ARENA SCRIPTING LANGUAGE DEVELOPER REFERENCE

NetModule Router SDK • Syntax, Runtime Architecture, Standard Library & Critical Implementation Notes

Language Version: 2.2 | **Library: 2.5.11** | **Sandbox: /mnt/sdk** | **Platform: NetModule NRSW**

ARCHITECTURE

SECTION 1: WHAT IS ARENA & CORE ARCHITECTURE

Arena is a lightweight, high-performance scripting language specifically designed for NetModule industrial routers (NRSW) and embedded systems. Heavily modeled on **ANSI C** syntax, it eliminates manual memory management and static typing friction while maintaining an ultra-compact interpreter footprint (under a few megabytes) suited for embedded devices.

Feature	ANSI C Paradigm	Arena Scripting Language (v2.2)
Memory Management	Manual (`malloc` / `free`). Vulnerable to memory leaks and overflows.	**Fully Automatic. Automatic reference tracking and garbage collection for all native types.**
Type System	Static typing (types fixed at compile time).	**Dynamic Typing. Variables can hold any datatype and change type dynamically during runtime.**
Variable Declaration	Mandatory explicit declaration before use (`int x;`).	**No Declaration Needed. Variables are automatically allocated upon first assignment.**
Data Structures & OOP	User-defined `typedef struct`, no classes or methods.	**Structure Templates (`template` / `extends`) providing class-like inheritance and methods.**
Constants	`const` keyword or preprocessor macros (`#define`).	**No Dedicated Keyword. Modeled cleanly via global variables written once at script initialization.**
Error & Exception	Error codes (`errno`, -1, NULL checks).	**Structured Exception Handling: Native `try { ... } catch (e) { ... }` and `throw`.**
Functional Constructs	Function pointers only.	**Anonymous Functions (Lambdas): Native first-class `\ (x) { ... }` expressions.**

SCOPES

1.2 Scopes and Namespaces

In Arena, namespaces govern the visibility and lifecycle of variables, functions, and templates:
• **Global Namespace:** Contains all entities defined at the script top-level. It is always visible throughout execution.
• **Local Namespace:** Automatically instantiated whenever a function or method is invoked. Local mutations do not affect the global namespace. Upon function return, the local namespace is destroyed and its allocated memory is automatically reclaimed.
• **`global()` and `unset()`:** Use `global("var_name");` inside a function to promote a local variable to global scope. Use `unset("var_name");` to delete entities from the active namespace.

SYNTAX

1.3 Code Organization (include) and Comments

• **include Directive:** Load external libraries using `include "library.inc";`. The file content is parsed directly into the current scope (guaranteed search in working directory). Preventing circular includes is the developer's responsibility.
• **Comments:** Single-line comments start with // or #. Multi-line block comments use /* ... */ (see Section 2 for critical apostrophe syntax rules in block comments).

IMPORTANT SECTION 2: CRITICAL IMPLEMENTATION NOTES & COMPILER PITFALLS

Developer Field Notice: Embedded Arena Guidelines

The practical rules below are compiled from real-world development on NetModule routers (NB2800, NB800, etc.). Strict adherence to these 6 rules will prevent the vast majority of silent compiler errors, unexpected lexer terminations, and router runtime 'job failed with rc 1' execution crashes.

PITFALL 1 1. Testing 'void' Values (Most Critical Syntax Rule)

ROOT CAUSE: In Arena, void is **NOT** a literal variable value; it is a reserved datatype keyword! Consequently, writing x == void or x != void triggers a fatal compiler crash: **'parse error, unexpected TYPE_VOID' (exit code 1)**.
CORRECT USAGE: Always test for void and undefined returns using the built-in type-testing functions:

```
// WRONG (Fatal Compiler Crash!)
if (x != void) { ... }
if (strstr(sender, "533") == void) { ... }
```

```
// CORRECT (Using Built-in Type Check Functions)
if (!is_void(x)) { ... }         // x is not void (value exists)
if (is_void(strstr(s, "533"))) { ... } // Substring not found (void returned)
// Applicable to: is_null(), is_int(), is_string(), is_array(), is_struct()
```

PITFALL 2 2. Single Quotes / Apostrophes (') Forbidden in Block Comments

ROOT CAUSE: Arena's lexer tokenizes characters aggressively. An apostrophe (') inside a block comment (/* ... */) is often misinterpreted as the opening delimiter of a single-quoted character or string constant. Failing to find a closing quote before the end of the file, the parser terminates fatally with: **'parse error, unexpected $' (EOF parse error)**.
RULE: Never use apostrophes in comments. Use single-line // comments or write apostrophe-free text.

```
/* Send response SMS's */  // FATAL: The lexer treats 's as an open string and crashes at EOF!
/* 'void' type check */    // FATAL: EOF parse error
```

```
/* Send response SMS */    // CORRECT: Free of apostrophes
// Send response SMS's     // SAFER: Single-line comment, but avoiding apostrophes is best practice
```

PITFALL 3 3. Array Instantiation: Square Brackets Are NOT Array Literals

ROOT CAUSE: Square brackets [ ] are strictly reserved for **element indexing** (e.g. arr[0]). Unlike Python, JavaScript, or modern languages, Arena does not support bracket-based array literals. Writing arr = [1, 2, 3]; produces **'parse error, unexpected ['**.
CORRECT USAGE: All arrays must be instantiated via the built-in **mkarray(...)** constructor function:

```
arr = [1, 2, 3];             // FATAL: parse error, unexpected '['
names = ["alice", "bob"];    // FATAL
```

```
arr = mkarray(1, 2, 3);       // CORRECT: Integer array
names = mkarray("alice", "bob"); // CORRECT: String array
empty = mkarray();            // CORRECT: Empty array with length 0
```

PITFALL 4 & 5 4. NetModule Job Modes: Event Trigger vs Daemon & Duplicate SMS

A) Event-Triggered Scripts (e.g. 'sms-received', 'wan-up'):
• The NetModule Event Manager launches the script automatically when the event occurs.
• **CRITICAL WARNING: NEVER include an infinite while (true) loop** in an event-triggered script!
• The script must process the queue and exit promptly via exit(0);. An infinite loop causes the router to spawn a new background daemon on every trigger event, leading to CPU exhaustion, memory exhaustion, and router freezes.
B) Continuous Daemon (Polling) Scripts: Configured on startup (`system-startup`), uses while (true) { ... sleep(3); }.
C) Preventing Duplicate SMS Loops: SMS entries in nb_sms_list() are persistent on the SIM/storage. Once processed, you **MUST explicitly call nb_sms_delete(msg_id)**; otherwise, the script will re-process the same SMS on every execution, generating infinite outbound messages and exhausting SIM balances!

PITFALL 6 5. Pre-Deployment Local Syntax Verification: 'arena -c'

Always validate your script syntax locally on your development machine before uploading to the router:
arena -c script_name.are
• If syntax is valid, it produces zero output and returns exit code 0.
• If syntax errors exist, it outputs the exact line and column number, preventing costly runtime job crashes on the hardware.

DATATYPES

SECTION 3: RUNTIME TYPE SYSTEM, EXPRESSIONS & OPERATORS

Arena provides 8 built-in datatypes. While user-defined primitive datatypes are not supported, the struct datatype can be extended via structure templates to achieve object-oriented polymorphism.

Datatype	Value Domain & Architectural Description	Example Literal Syntax
void	Represents the absence of a meaningful value. Exactly one value exists.	`()` (two adjacent parentheses - unit)
bool	Boolean truth values for logical operations.	`true`, `false`
int	Signed integer with at least 32-bit precision (-2^31 to 2^31 - 1). No unsigned integers.	`42`, `-100`, `0xFF` (hexadecimal)
float	IEEE double-precision floating-point number.	`3.14159`, `-0.005`, `1.2e3`
string	Binary-safe byte sequence. **Unlike C, strings may contain embedded `\0` (null) bytes.**	`"Hello\n"`, `'Single quoted'`
array	Heterogeneous indexed list. 0-indexed. **Supports negative indexing (`-1` is last element).**	`mkarray(10, "node", 4.5)`
struct	Collection of fields and methods indexed by string identifiers. Mimics OOP objects.	`mkstruct("id", 1, "host", "gw")`
fn	First-class function value. Can be passed as arguments or returned from functions.	`\ (x) { return x * 2; }` (lambda)
resource	Opaque operating system resources (file handles, sockets, allocated memory, PCRE).	`fopen("/tmp/data", "r")`

CASTING

3.2 Type Conversions (Cast Expressions)

Arena supports both explicit casting (`(int) "42"`) and automatic context-driven coercion:
- **Conversion to bool:** `0`, `0.0`, `""` (empty string), empty array `mkarray()`, empty struct, and `()` (void) evaluate to `false`. All other values evaluate to `true`.
- **Conversion to string:** Numbers convert to decimal text; arrays and structures format into string representations.
- **Conversion to array:** Casting a non-array value creates a 1-element array holding that value: `(array) 5` -> `mkarray(5)`.
- **Conversion to struct:** Casting a scalar value creates a struct with a single field named `"value"`: `(struct) 42` -> `st.value == 42`.
- **Conversion to fn / resource:** Attempting to cast an incompatible type to `fn` or `resource` is a **fatal runtime error**.

OPERATORS

3.3 Operators, Precedence & Special Expressions

Category	Operators	Associativity / Precedence	Description & Usage
Cast	`(type) expr`	Highest / Right	Explicit datatype conversion (`(int) x`)
Power	`**`	High / Right	Mathematical exponentiation: `2 ** 8 == 256`
Unary	`++`, `--`, `~`, `!`	High / Right	Increment, decrement, bitwise NOT, logical NOT
Multiplicative	`*`, `/`, `%`	Medium-High / Left	Arithmetic multiplication, division, modulo
Additive	`+`, `-`	Medium / Left	Arithmetic addition and subtraction
Shift	`<<`, `>>`	Medium / Left	Bitwise left and right shift
Relational	`<`, `<=`, `>`, `>=`	Medium-Low / Non-assoc	Comparison evaluations
Equality	`==`, `!=`	Medium-Low / Non-assoc	Type and value equality checks
Bitwise	`&`, `^`, `|`	Low / Left	Bitwise AND, XOR, OR
Logical	`&&`, `||`	Very Low / Right	Short-circuit boolean AND, OR
Ternary	`? :`	Very Low / Non-assoc	Conditional operator: `guard ? a : b`
Assignment	`=`, `+=`, `-=`, `*=`, `/=`, `&=`, `|=`, `^=`, `<<=`, `>>=`	Lowest / Right	Value assignment and compound operations (chainable: `a = b = 0`)

- **Negative Indexing:** `arr[-1]` accesses the last array element; `arr[-2]` accesses the second to last.
- **Deep Auto-Vivification:** Writing `a.b[2].c = 5;` automatically constructs nested structures and arrays if uninitialized.
- **Built-in Constants:** `__FILE__` evaluates to the script filename (string); `__LINE__` evaluates to current line number (int).

SYNTAX

SECTION 4: CONTROL FLOW, FUNCTIONS & OOP TEMPLATES

FLOW

4.1 Control Flow Statements & Exception Handling

• **if / else:** The guard expression automatically casts to `bool`: `if (x % 2 == 0) { ... } else { ... }`
• **Loops (while, do-while, for):** Identical to ANSI C semantics. `while (guard) { ... }`, `do { ... } while (guard);` (guaranteed at least one execution). `for (i=0; i<10; i++) { ... }`. Infinite loop: `for (;;) { ... }` or `while (true)`.
• **switch / case / default:** Matches guard value and type. **Without an explicit `break`, execution falls through:**
switch (val) { case 0: ++a; case 1: ++b; break; default: ++c; }
• **Exception Handling (try / catch / throw):** Structured error management. Any datatype can be thrown:
try { if (err) throw "Critical Error"; } catch (e) { printf("Caught: %s\n", e); }

FUNCTIONS

4.2 User-Defined Functions & Pass-By-Reference

• **Function Definition:** Defined at the global script level; function definitions cannot be nested.
• **The `forced` Specifier:** Prefixing an argument or return type with `forced` automatically casts incoming values to the specified type. Without `forced`, passing an incompatible type triggers a fatal error:
int add(forced int x, forced int y) { return x + y; } // add("10", 20) safely returns 30
• **The `mixed` Specifier:** Denotes that a parameter or return value can hold arbitrary datatypes.
• **Pass-By-Reference (Atomic Copy-In Copy-Out):** Invoked by prefixing an argument with &. Arena does not use raw pointers; values are copied into local parameters upon entry (copy-in), and atomically copied back to caller variables upon function completion (copy-out):
void swap(mixed a, mixed b) { c = a; a = b; b = c; } // Call: swap(&x, &y);

OOP

4.3 Structure Templates & Object-Oriented Inheritance

Object-oriented programming in Arena is implemented via **`template`** and **`extends`** keywords. Templates act as blueprints to generate struct instances endowed with data fields and methods:
• **Constructor Method:** A method sharing the exact same identifier as the template. Executes automatically upon `new`.
• **The `this` Variable:** A local struct copy available inside methods representing the current instance.
• **Static Reference:** Access template members without instantiation using the double colon operator (Template::member).

```
template Device {
    name = "Generic";
    void Device(string n) { this.name = n; }
    void describe() { printf("Device: %s\n", this.name); }
}
template Router extends Device {
    void Router(string n) { this.name = n; }
    void describe() { printf("Router Node: %s (Online)\n", this.name); }
}
r = new Router("NB2800-GW");
r.describe(); // Output: Router Node: NB2800-GW (Online)
```

LAMBDA

4.4 Anonymous Functions (Lambda Expressions)

For ad-hoc, throwaway logic, anonymous functions can be constructed on the fly using a leading backslash \. Borrowed from Haskell syntax, lambdas integrate seamlessly with higher-order functions like `map`, `filter`, and `foldl`:

```
square = \ (x) { return x * x; };
nums = mkarray(1, 2, 3, 4);
squares = map(\ (x) { return x * x; }, nums); // Evaluates to [1, 4, 9, 16]
```

LIBRARY

SECTION 5: STANDARD LIBRARY REFERENCE - PART 1 (v2.5.11)

RUNTIME

5.1 Runtime System & Type Introspection

Function Prototype	Returns	Functional Description & Behavior
`type_of(mixed x)`	`string`	Returns typename string: `"void"`, `"int"`, `"string"`, `"array"`, `"struct"`, etc.
`tmpl_of(mixed x)`	`mixed`	Returns the template name from which the struct was constructed; otherwise `()`.
`is_a(mixed x, string type)`	`bool`	Tests whether value matches the indicated type or inherits from the specified template.
`is_void(x, ...)`, `!is_void(x)`	`bool`	**Tests whether arguments are void (the indispensable, safe void check).**
`is_bool`, `is_int`, `is_float`	`bool`	Verifies whether argument is a boolean, integer, or floating-point value.
`is_string`, `is_array`, `is_struct`	`bool`	Verifies whether argument is a string, array, or struct value.
`is_fn`, `is_resource`	`bool`	Verifies whether argument is a first-class function (`fn`) or system `resource`.
`cast_to(mixed x, string type)`	`mixed`	Dynamically casts value `x` to the designated datatype name.
`set(name, val)` / `get(name)`	`bool` / `val`	Dynamically defines or retrieves a variable in the active namespace by name.
`unset(forced string name, ...)`	`void`	Removes entities from local namespace (or global if called at top-level scope).
`global(forced string name, ...)`	`void`	Promotes local variables into the global namespace.
`assert(forced bool cond, ...)`	`void`	Halts script immediately with 'assertion failure' if any condition evaluates to false.

STRINGS

5.2 String Manipulation Functions

Function Prototype	Returns	Functional Description & Behavior
`strlen(forced string s)`	`int`	Returns total character / byte length of the string.
`strcat(mixed x, ...)`	`string`	Converts all arguments to string and returns their concatenation.
`strstr(hay, needle)`	`mixed`	Searches for substring `needle` in `hay`; returns index or `()` if absent.
`strchr(hay, needle)` / `strrchr`	`mixed`	Locates leftmost or rightmost character occurrence; returns index or `()`.
`substr(s, pos, [max])`	`string`	Extracts substring starting at index `pos` up to optional `max` length.
`left(s, max)` / `right(s, max)`	`string`	Returns the leftmost or rightmost `max` characters.
`trim(s)` / `ltrim` / `rtrim`	`string`	Strips whitespace characters from both ends, leading end, or trailing end.
`tolower(s)` / `toupper(s)`	`string`	Converts all alphabetic characters to lowercase or uppercase.
`explode(forced string s)`	`array`	Splits string into an array of single-character string elements.
`implode(array arr)`	`string`	Concatenates all array elements into a unified string value.
`ord(s)` / `chr(int code)`	`int` / `str`	Returns ASCII code of first character or constructs single-char string from ASCII code.

ARRAYS

5.3 Array & Higher-Order List Operations

Function Prototype	Returns	Functional Description & Behavior
`mkarray(mixed x, ...)`	`array`	**Constructs an array. Use `mkarray()` for empty arrays (never `[]`).**
`length(array arr)`	`int`	Returns the number of elements in the array.
`qsort(array arr)` / `is_sorted`	`array` / `bool`	Sorts array copy using quicksort or verifies sorted order.
`array_search(hay, needle)`	`mixed`	Finds matching element using `==` semantics; returns int index or `()`.
`array_merge(x, ...)`	`array`	Merges multiple arrays into a single composite array.
`array_compact(array arr)`	`array`	Removes all `void` (`()`) elements, compacting the array to minimum size.
`head(arr)` / `tail(arr)`	`mixed` / `arr`	Returns first element (`head`) or returns remainder after removing first (`tail`).
`take(arr, n)` / `drop(arr, n)`	`array`	Takes first `n` elements or discards first `n` elements.
`map(fn f, array arr, ...)`	`array`	Applies function `f` to every element, collecting results into a new array.
`filter(fn f, array arr, ...)`	`array`	Filters array elements retaining only those where `f(elem)` evaluates to `true`.
`foldl(f, init, arr)` / `foldr`	`mixed`	Reduces array left-to-right (`foldl`) or right-to-left (`foldr`) into a single value.

STRUCTS

5.4 Structure (Struct) Functions

Function Prototype	Returns	Functional Description & Behavior
`mkstruct(k1, v1, k2, v2, ...)`	`struct`	Constructs a struct from alternating key-value pairs.
`struct_get(st, "key")`	`mixed`	Retrieves field value by key name string; returns `()` if missing.
`struct_set(st, "key", val)`	`bool`	Inserts or updates field value in struct.
`struct_fields(struct st)`	`array`	Returns string array of all data field names (excluding methods).
`struct_methods(struct st)`	`array`	Returns string array of all method names (fields of type `fn`).
`struct_merge(st1, st2, ...)`	`struct`	Merges multiple structs; fields in later arguments overwrite earlier ones.

SYSTEM

SECTION 6: FILE I/O, C INTEROPERABILITY (FFI) & ROUTER SDK API

FILES

6.1 File Input/Output & The SDK Sandbox (/mnt/sdk)

On NetModule routers, all SDK file I/O operations execute inside the **`/mnt/sdk` sandbox**. Files written to `/tmp` reside in volatile memory and are cleared upon script reboots.

Function Prototype	Modes / Parameters	Functional Description & Behavior
`fopen(name, mode)`	`"r"`, `"w"`, `"a"`, `"r+"`, `"w+"`, `"a+"`	Opens file and returns file `resource`. Returns `()` on failure or invalid mode.
`fclose(resource handle)`	File resource handle	Closes open file, flushes dirty buffers to disk, and releases OS resource.
`fread(h, max)` / `fwrite(h, str)`	`max` bytes / `data` string	Reads binary/text data or writes string to file; returns bytes processed.
`fgets(resource handle)`	Single line read	Reads characters up to newline (`\n`) or EOF; returns string or `()` on EOF.
`fseek(h, pos)` / `ftell(h)`	`pos` (+ start, - end)	Reposition file pointer or returns current byte offset.
`feof(h)` / `ferror(h)` / `clearerr(h)`	File resource handle	Tests end-of-file condition, queries error indicator, or clears errors.

FFI

6.2 Foreign Function Interface (C FFI) & Memory Buffers

Arena can dynamically link shared C libraries (`.so`) and invoke native C functions directly. Memory resources are used to bridge C pointers and raw byte buffers:

- **`dyn_supported()`:** Queries whether dynamic library loading is enabled on the architecture.
- **`dyn_open(name)` / `dyn_close(lib)`:** Loads `.so` shared object / unloads library.
- **`dyn_call_void`, `dyn_call_int`, `dyn_call_float`, `dyn_call_ptr`:** Dispatches native C function calls.
- **Memory Functions:** `malloc(size)`, `calloc(n, size)`, `free(mem)`, `cstring(str)`, `mstring(mem, off)`, `mputint(mem, off, val)`, `mgetint(mem, off)`, `memcpy(dst, d_off, src, s_off, len)`.

SDK API

6.3 NetModule Router Hardware & Service SDK APIs

API Domain	Example Signatures	Operational Description & Parameters

System Status (`nb_status`)	`nb_status("location")`
	`nb_status("wan")`
	`nb_status("system")`
		Retrieves GPS coordinates (lat, lon, city), cellular network signal strength, or router serial number as a populated struct.

Configuration	`nb_config_get("wan.0.mode")` `nb_config_set("wan.0.mode=dhcp")`	Live queries or alters router configuration parameters. Changes take effect dynamically across the operating system.
Digital I/O (DIO)	`nb_dio_get(int port)` `nb_dio_set(int port, int val)`	Reads input port states (0/1) or toggles relay output pins to high (1) or low (0).

SMS Service	`nb_sms_list()`
	`nb_sms_send(phone, msg)`
	`nb_sms_delete(msg_id)`
		Lists inbox messages, sends outbound SMS. **Crucial: Processed SMS must be deleted via `nb_sms_delete` to prevent infinite loops!**

Sockets & Network	`nb_tcp_server(...)` `socket()`, `bind()`, `connect()`	Creates persistent TCP/UDP servers, manages raw socket communication, and monitors multiplexed file descriptors with select.

System Utilities	`sleep(sec)`, `usleep(usec)`
	`json_encode()`, `md5sum()`
	`nb_syslog("notice message")`
		Executes timer sleeps, encodes data into JSON, calculates MD5 hashes, and writes structured alerts directly to system logs.

NetModule Deployment Notice

Deployment Tip: Upload production `.are` scripts via NetModule Web Manager under *SDK -> Scripts*, and configure corresponding scheduled or event-driven jobs under the *Jobs* administration menu.